



# ichosynth — Development Environment

## Complete, up-to-date guide for Windows and macOS

Everything you need to build, modify, and upload the firmware: from the toolchain to the libraries, all the way to your first upload onto the Teensy 4.1.

Updated June 2026

arduino-cli required

teensy:avr ≥ 1.61.0

OS Windows · macOS

⚡ **In a hurry?** ichosynth runs the real **TœRN** firmware, and one command builds it for our hardware: `python teensy/build_toern.py`. Jump to [chapter 3](#). The rest of the guide explains every step and prerequisite.

### Table of Contents

- 1 · What we install
- 2 · Requirements
- 3 · The fast track: one command builds it all
- 4 · Installing the toolchain (arduino-cli)
- 5 · Adding Teensy support (the teensy:avr core)
- 6 · The libraries
- 7 · How the build script works
- 8 · Building and uploading
- 9 · Building from the Arduino IDE (alternative)
- 10 · Optional tools (Python, Git)
- 11 · Checklist and common problems

## 1 · What we install

ichosynth is a **Teensy 4.1** running the real **TœRN** firmware (by SP\_ / soundpauli — <https://toern.live>) on cheap, hand-soldered parts. We don't keep a separate copy of TœRN in the repo: a build script fetches the upstream sources and compiles them for our hardware. Here's what you need:

| Component       | Version  | What it's for                                              | Required |
|-----------------|----------|------------------------------------------------------------|----------|
| arduino-cli     | latest   | the command-line build toolchain                           | ✓        |
| teensy:avr core | ≥ 1.61.0 | the Teensy compiler + Teensy Loader                        | ✓        |
| Python          | 3.x      | runs <code>teensy/build_toern.py</code> (the build script) | ✓        |

| Component      | Version     | What it's for                                                          | Required    |
|----------------|-------------|------------------------------------------------------------------------|-------------|
| Git            | latest      | the script clones the TCERN sources on first run                       | ✓           |
| Custom drivers | in the repo | i2cEncoderLibV2 , FastTouch , Ichos01ed — already in teensy/libraries/ | ✓ (bundled) |
| GitHub CLI     | latest      | optional, for version-controlling the project                          | optional    |

⚙️ **The one thing that bites everyone:** TCERN is a single ~23k-line translation unit, and the Teensy gcc **crashes at the default -O2** (exits 0xffffffff — an internal compiler error / out-of-memory). The fix is to compile at **-O1** ( `opt=o1std` ). The build script sets this for you; if you build from the Arduino IDE instead, you must pick **Tools** → **Optimize** → **"Fast"**. See [chapter 7](#).

## 2 · Requirements

- **Windows:** Windows 10 or 11, 64-bit.
- **macOS:** macOS 10.15 or later; native support for **Intel** and **Apple Silicon** (M1/M2/M3/M4).
- **A micro-USB data cable** (not charge-only) for the Teensy 4.1.
- A **Teensy 4.1 with 16 MB PSRAM** (both chips) soldered on — TCERN needs the PSRAM for its sample buffers.
- ~1 GB of disk space for the toolchain + core + libraries.

## 3 · The fast track: one command builds it all

Once you have **arduino-cli**, the **teensy:avr core**, **Python**, and **Git** (chapters 4–5), building the firmware is a single command from the repo root:

```
python teensy/build_toern.py
```

This produces the firmware at `teensy/firmware/toern.hex` . To build *and* flash in one go (if `teensy_loader_cli` is on your PATH):

```
python teensy/build_toern.py --flash
```

Useful flags:

| Flag                 | Effect                                                                      |
|----------------------|-----------------------------------------------------------------------------|
| <code>--flash</code> | after building, upload with <code>teensy_loader_cli</code>                  |
| <code>--keep</code>  | keep the temporary build folder (for inspecting the staged/patched sources) |

💡 The **first** build is slow (the Teensy core is compiled once) and it also **clones the TCERN sources** into `emulator/toern-src/` if they aren't there yet. Later builds are fast.

🔔 Just want to flash a **prebuilt** `.hex` ? There's a one-click GUI flasher in `_FLASHER/` — see [chapter 8.3](#).

## 4 · Installing the toolchain (arduino-cli)

We build from the command line with `arduino-cli`.

### Windows

```
winget install ArduinoSA.CLI
```

### macOS

```
brew install arduino-cli
```

Check it's on your PATH:

```
arduino-cli version
```

**i Drivers:** on Windows 10/11 and on macOS **no additional drivers are needed** for the Teensy 4.1.

## 5 · Adding Teensy support (the teensy:avr core)

The Teensy compiler and Teensy Loader come from the **teensy:avr core**. Add PJRC's package index, update, and install the core (we need **version ≥ 1.61.0**):

```
arduino-cli config init
arduino-cli config add board_manager.additional_urls https://www.pjrc.com/teensy/package_teensy_index.json
arduino-cli core update-index
arduino-cli core install teensy:avr
```

Check the version you got:

```
arduino-cli core list
```

You should see `teensy:avr` at **1.61.0** or newer.

💡 Prefer a GUI? You can instead install **Teensyduino 1.61** through the Arduino IDE 2.x Boards Manager (PJRC's package URL above) — that installs the same `teensy:avr` core. Either way, the build script just needs the core present

and `arduino-cli` on your PATH.

## 6 · The libraries

The TCERN firmware compiles against three **custom drivers** that adapt it to our cheap hardware. They are **already in the repo** under `teensy/libraries/`, and the build script points `arduino-cli` at them automatically — you don't install them:

| Library                | Replaces                                                                                    | Where the hardware config lives                               |
|------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>i2cEncoderLibV2</b> | TCERN's 4 I <sup>2</sup> C RGB encoders → our 4 KY-040 mechanical encoders                  | <code>ICHOS_ENC_PINS</code> in <code>i2cEncoderLibV2.h</code> |
| <b>FastTouch</b>       | TCERN's 3 capacitive touch pads → our 3 momentary tact switches                             | <code>ICHOS_BTN_PINS</code> in <code>FastTouch.h</code>       |
| <b>IchosOled</b>       | adds the I <sup>2</sup> C <b>OLED HUD</b> (the state TCERN showed on the encoder RGB rings) | <code>teensy/libraries/IchosOled/</code>                      |

 **Where the hardware lives:** pin assignments live in those two headers ( `ICHOS_ENC_PINS` , `ICHOS_BTN_PINS` ) and in `teensy/build_toern.py` — **not** in a `config.h` . ( `config.h` belongs to the old NI404 fallback build and is not used by the TCERN port.)

The build script also resolves any remaining stock libraries (the **Audio**, **Wire**, **SD**, etc. bundled in the Teensy core, plus anything in your user `Arduino/libraries/` ) the way `arduino-cli` normally does.

## 7 · How the build script works

`teensy/build_toern.py` compiles the **unmodified** TCERN sketch, applying only the minimal changes that make it run on our hardware. Step by step:

1. **Locate the sources.** It looks for the TCERN sketch under `emulator/toern-src/` ; if it isn't there, it **clones it** from upstream automatically (so the repo stays free of a vendored copy).
2. **Stage them** into a temporary build folder (the originals stay pristine).
3. **Pin remap.** TCERN's `SWITCH_1/2/3` default to pins that collide with our right-hand KY-040, so they are remapped to **25 / 26 / 28** — the same pins as `ICHOS_BTN_PINS` in `FastTouch.h` .
4. **Feature trim.** It removes the optional **2nd LED strip** (frees a pin and saves CPU), matching our build.
5. **OLED HUD.** It wires in the `IchosOled` library (include + `begin()` + a one-line render call in `loop()` ) so the status HUD compiles as its own translation unit.
6. **Compile** with `arduino-cli` for the FQBN below and copy the result to a stable name, `toern.hex` .

The exact target it builds for is:

```
teensy:avr:teensy41:usb=serialmidi16,opt=o1std
```

- `usb=serialmidi16` → **USB type Serial + MIDIx16** (so `usbMIDI` works).

- `opt=01std` → `-01` . This is the critical flag: the default `-02` crashes the Teensy gcc on TCERN's giant single translation unit. `-01` builds cleanly and the firmware still fits with room to spare.

---

## 8 · Building and uploading

### 8.1 Build

From the repo root:

```
python teensy/build_toern.py
```

When it finishes you'll have `teensy/firmware/toern.hex` .

### 8.2 Build and flash in one step

If you have `teensy_loader_cli` on your PATH:

```
python teensy/build_toern.py --flash
```

1. Connect the Teensy 4.1 via USB (a **data** cable).
2. The loader usually programs on its own; if it stays waiting, press the **little white button** on the Teensy.
3. When the flash finishes, the animation starts on the LED matrix.

### 8.3 Flashing the firmware

If you'd rather flash a **prebuilt** `.hex` (or don't have `teensy_loader_cli` ), use the **one-click GUI flasher** in `_FLASHER/` — point it at `teensy/firmware/toern.hex` and click flash.

---

## 9 · Building from the Arduino IDE (alternative)

You can also build TCERN by hand in the Arduino IDE 2.x — useful for stepping through the code. You'll need to reproduce what the script does:

| Tools menu item | Value                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------------|
| Board           | Teensy 4.1                                                                                         |
| USB Type        | Serial + MIDIx16                                                                                   |
| CPU Speed       | 600 MHz (default)                                                                                  |
| Optimize        | "Fast" ⚠️ (this is <code>-01</code> ; the default "Faster"/ <code>-02</code> crashes the compiler) |
| Port            | the Teensy's port (after connecting it)                                                            |

You'll also have to apply the pin remap ( SWITCH\_1/2/3 → 25/26/28), make the three custom libraries in teensy/libraries/ visible to the IDE, and wire in the OLED HUD by hand. **The script does all of this for you**, which is why it's the recommended path.

 Don't forget **Optimize** → "**Fast**". If you leave it at the default, the build dies with an internal compiler error ( exit 0xffffffff ) partway through TCERN's single huge file.

## 10 · Optional tools (Python, Git)

### 10.1 Python

The build script needs **Python 3**. (Git is also required — see below — because the script clones the TCERN sources on first run.)

**Windows:** winget install Python.Python.3.12 · **macOS:** brew install python@3.12

 Any modern Python 3 works for the build script. (If you also use the wavmaker sample converter and run it from source on **Python 3.13+**, that one needs pip install audioop-1ts , because audioop was removed from the standard library — but the ready-made wavmaker.exe in \_SDCARD/ needs no Python at all.)

### 10.2 Git and GitHub CLI

Git is required (the script clones TCERN). GitHub CLI is optional.

**Windows:** winget install Git.Git (+ GitHub.cli optional) **macOS:** xcode-select --install (includes git) · brew install gh (optional)

## 11 · Checklist and common problems

| ✓                        | Check                                                                     |
|--------------------------|---------------------------------------------------------------------------|
| <input type="checkbox"/> | arduino-cli version works (it's on your PATH)                             |
| <input type="checkbox"/> | arduino-cli core list shows <b>teensy:avr ≥ 1.61.0</b>                    |
| <input type="checkbox"/> | <b>Python 3</b> and <b>Git</b> are installed                              |
| <input type="checkbox"/> | python teensy/build_toern.py produces teensy/firmware/toern.hex           |
| <input type="checkbox"/> | (IDE build only) USB Type = Serial + MIDIx16 <b>and</b> Optimize = "Fast" |
| <input type="checkbox"/> | Teensy 4.1 has <b>16 MB PSRAM</b> soldered                                |

| Symptom                                                                  | Cause / fix                                                                                                 |
|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>arduino-cli</code> not on PATH ... from the script                 | install <code>arduino-cli</code> (ch. 4) and reopen the terminal                                            |
| Compile dies with <code>exit 0xffffffff</code> / internal compiler error | you built at <code>-02</code> ; use the script, or set IDE Optimize → "Fast" ( <code>-01</code> )           |
| <code>teensy:avr</code> missing / too old                                | run <code>arduino-cli core install teensy:avr</code> and check <code>core list</code> (need $\geq 1.61.0$ ) |
| Script can't clone the TCERN sources                                     | install Git, or clone manually into <code>emulator/toern-src/</code>                                        |
| 'usbMIDI' was not declared (IDE build)                                   | USB Type not set to Serial + MIDIx16                                                                        |
| <code>teensy_loader_cli</code> not found ON <code>--flash</code>         | use the GUI flasher in <code>_FLASHER/</code> instead                                                       |
| Teensy not programming                                                   | press the <b>white button</b> on the board; or use a <b>data</b> USB cable, not charge-only                 |
| Port not visible                                                         | charge-only USB cable → use a data cable                                                                    |

Part of the *ichosynth* project · runs the real **TCERN** (SP\_) firmware · for the ICHOS 2026 workshop.